Лабораторна робота 7. Компоненти та інтерактивний каталог у Blazor

Завдання: власний каталог із налаштуванням товарів

Створіть власний застосунок на **Blazor Web App, .NET 10, Interactive Server**. Користувач повинен бачити щонайменше шість товарів, відкривати налаштування, змінювати два параметри та кількість, бачити перерахунок ціни й підтверджувати або скасовувати вибір.

Результат підтвердження показуйте під каталогом: назва, опції, кількість і сума. Повторне підтвердження замінює попередній результат. **У цій роботі ще немає кошика, сервісу стану, EF Core та SQLite.** Каталог і останній вибір містяться в пам'яті компонента.

Campus Café — наскрізний приклад, а не обов'язкова тема. Готовий демопроєкт показує результат двох робіт; у цій роботі виконуйте лише наведені тут кроки. Зберігайте власні назви й формулу під час усіх наступних замін коду.

Варіанти тем

Тема	Приклади товарів каталогу	Параметр 1	Параметр 2	Що впливає на ціну
Майстерня букетів	Букет троянд, тюльпанів, сезонний мікс	Кількість квітів: 5, 9, 15	Пакування: звичайне / подарункове	Доплата за кількість квітів і пакування
Друк фотографій	Фото на матовому папері, на глянці, фотолистівка	Формат: 10×15, 15×21, 20×30	Обробка: без рамки / з рамкою	Доплата за формат і рамку
Майстерня подарункових наборів	Чайний набір, солодкий набір, набір для творчості	Розмір: міні / стандарт / великий	Пакування: коробка / подарункова скринька	Доплата за розмір і пакування
Сендвіч-бар	Овочевий, курячий, сирний сендвіч	Розмір: звичайний / великий	Додаток: без додатка / додатковий сир	Доплата за розмір і додаток
Друк на одязі	Футболка, худі, світшот різних моделей	Розмір одягу: S, M, L	Друк: з одного / з двох боків	Доплата за друк; розмір може не змінювати ціну
Магазин зошитів і блокнотів	Лінійка, клітинка, скетчбук, планер	Кількість сторінок: 48, 96, 144	Обкладинка: м'яка / тверда	Доплата за сторінки й обкладинку

Можна обрати також іншу тематику, відсутню в таблиці вище, за бажанням.

Для будь-якої теми **кількість одиниць замовлення — окреме поле від 1 до 10**. Наприклад, «9 квітів у букеті» та «2 букети» — різні параметри.

Перед початком коротко запишіть: назву застосунку; шість товарів; два параметри налаштування; базові ціни й доплати; формулу вартості. Розрахуйте вручну один приклад — він стане контрольним тестом.

Що здати

- Проєкт власної тематики: щонайменше шість товарів, два параметри, кількість 1–10, принаймні одна платна опція.
- Окремі компоненти картки, діалогу та форматування ціни; використання `[Parameter]`, `EventCallback` і `@bind`.
- Власні зображення, назви, підписи й базове оформлення.
- Три ручні розрахунки з очікуваними сумами та коротку інструкцію запуску. Код передавайте без `bin` і `obj`.

Мета: навчитися відділяти модель від інтерфейсу, передавати дані вниз по дереву компонентів і повідомлення — назад до батьківського компонента.

У лістингах використано простір імен `CampusCafe`. Якщо ваш проєкт має іншу назву, узгодьте `namespace`, `@using` і назву `*.styles.css`. Перейменовуйте типи через Rename редактора, а не лише підписи на сторінці.

Як читати лістинги зі змінами

Формат лістинга залежить від обсягу роботи:

- **Новий файл** — повний вміст у блоці `csharp` або `razor`.
- **Кілька локальних змін** — короткий `diff` із контекстом.
- **Значна перебудова** — спочатку повний старий вміст («Було»), нижче повний новий («Стало»).
- **Оформлення CSS** — лише новий повний вміст файла.

У блоці `diff` позначення читаються так:

- `-` на початку рядка — вилучіть цей рядок.
- `+` на початку рядка — додайте цей рядок без початкового знака `+`.
- Пробіл на початку — незмінний контекст, який допомагає знайти потрібне місце.
- `--- a/...` і `+++ b/...` — назва файла до та після зміни; це не рядки програми.
- `@@ ... @@` — межі фрагмента і номери рядків; їх не потрібно копіювати у файл.

Не замінюйте весь файл одним фрагментом diff. Застосовуйте зміни послідовно: кожен diff порівнює код із попереднім етапом цієї роботи, а не завжди з початковим шаблоном. Пропущені між фрагментами рядки залишаються без змін. Пробіл контексту та службові позначення diff не належать до коду; відступи самого коду потрібно зберегти.

У різних виправленнях SDK можуть відрізнятися коментарі й форматування шаблону. Орієнтуйтеся на назви файлів і навколишній код, а не лише на номери рядків.

1. Середовище та створення проєкту

Потрібні .NET SDK 10, редактор коду та браузер. Для початкового відновлення NuGet-пакетів потрібен інтернет.

Перевірте середовище:

```
dotnet --version
dotnet --list-sdks
dotnet --list-runtimes
```

Має бути встановлено SDK гілки `10.0` і середовище виконання `Microsoft.AspNetCore.App 10.0`. Якщо команда `dotnet` не знайдена, установіть **SDK**, а не лише Runtime, і відкрийте нове вікно термінала.

Адаптуйте зараз — назва проєкту. Оберіть назву папки й простору імен, якщо хочете відразу використовувати власну. У команді нижче `CampusCafe` — назва демонстрації. Далі не змішуйте простори імен двох різних проєктів.

У папці для навчальних проєктів виконайте:

```
dotnet new blazor -n CampusCafe -f net10.0 --interactivity Server --all-interactive --empty
--no-https
cd CampusCafe
dotnet run --urls http://localhost:5127
```

Пояснення параметрів:

Параметр	Для чого потрібний
`blazor`	Новий шаблон Blazor Web App.
`-n CampusCafe`	Назва проєкту й базового простору імен.
`-f net10.0`	Цільова платформа .NET 10.
`--interactivity Server`	Інтерактивний C# виконується на сервері.
`--all-interactive`	Інтерактивний режим застосовується до дерева маршрутизованих сторінок.
`--empty`	Не додавати навчальні сторінки Counter і Weather та їх оформлення.
`--no-https`	Локальна вправа використовує HTTP без налаштування сертифіката. Для публічного розгортання потрібний HTTPS.

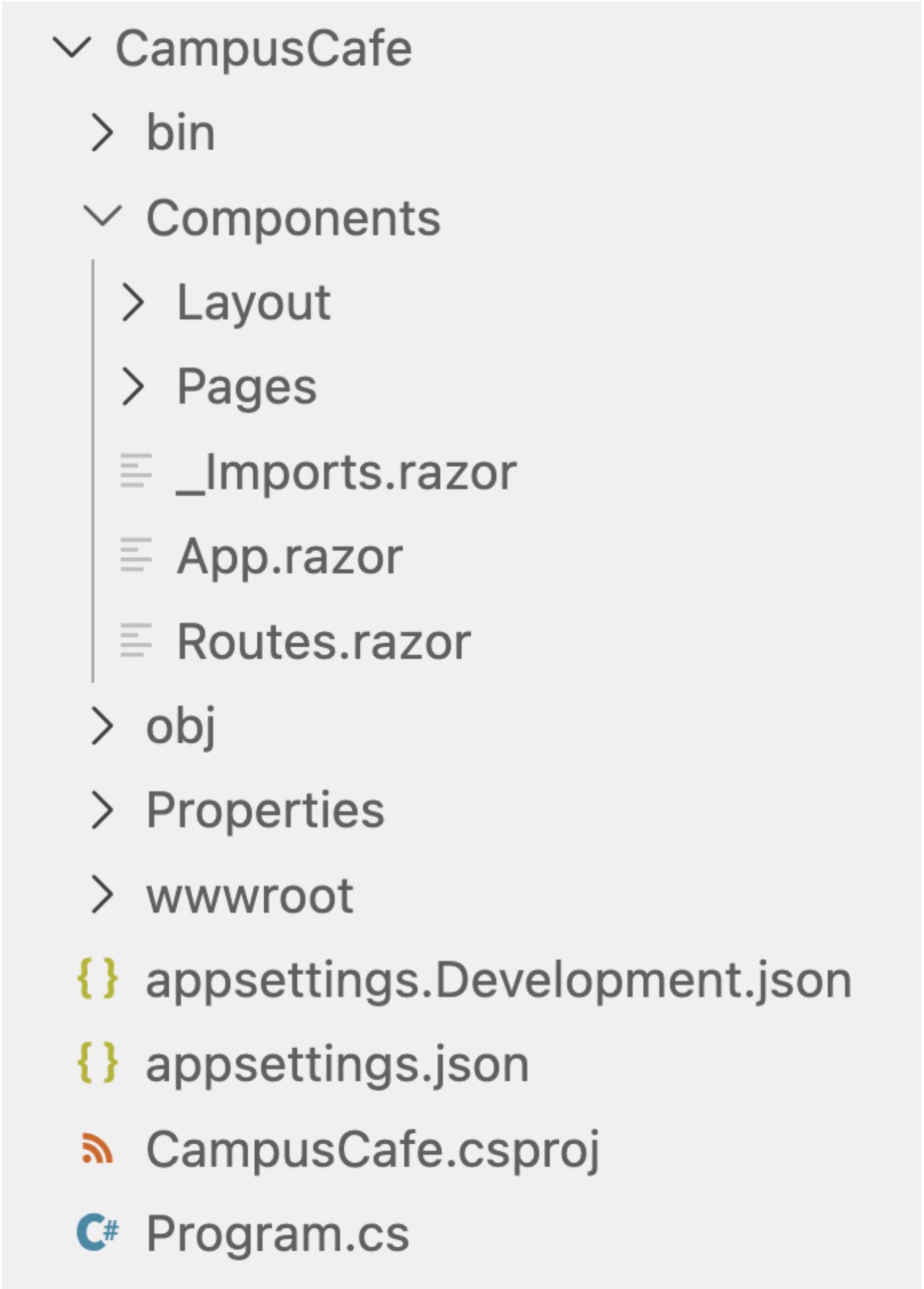
Відкрийте `http://localhost:5127`. Має з'явитися **Hello, world!**. Зупинка сервера в терміналі — `Ctrl+C`. Далі команди виконуйте з папки, де лежить `CampusCafe.csproj`.

Якщо порт зайнятий, використайте, наприклад, `--urls http://localhost:5128` і відповідну адресу браузера. Не запускайте два екземпляри на одному порту. Після змін перезапускайте `dotnet run` або використовуйте `dotnet watch --urls http://localhost:5127`.

У Visual Studio можна створити **Blazor Web App**, вибрати .NET 10, режим Server і глобальну інтерактивність. Для точного відтворення структури цієї роботи використовуйте наведену команду, а потім відкрийте `.csproj` у редакторі.

2. Початкова структура і Program.cs

Основні файли шаблону:


CampusCafe
bin
Components
Layout
Pages
_Imports.razor
App.razor
Routes.razor
obj
Properties
wwwroot
appsettings.Development.json
appsettings.json
CampusCafe.csproj
Program.cs

`Program.cs` запускає вебзастосунок. `App.razor` задає HTML-оболонку, `Routes.razor` знаходить компонент за адресою, а `MainLayout.razor` додає спільну шапку й підвал. Файли `.razor` — компоненти, а не Razor Pages з `.cshtml.cs`.

Початковий `Program.cs` має таку логіку:

```
using CampusCafe.Components;

// Створення об’єкта для налаштування застосунку.
var builder = WebApplication.CreateBuilder(args);

// Реєстрація Razor-компонентів і підтримки інтерактивності на сервері.
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

// Побудова застосунку після реєстрації сервісів.
var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
}

// Сторінка для відповідей на зразок 404.
app.UseStatusCodePagesWithReExecute("/not-found", createScopeForStatusCodePages: true);
// Підтримка перевірки anti-forgery токенів там, де вона потрібна.
app.UseAntiforgery();

// CSS, JavaScript та інші статичні ресурси.
app.MapStaticAssets();

// Підключення кореневого компонента та режиму Interactive Server.
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

// Запуск сервера; він працює до завершення процесу.
app.Run();
```

Реєстрація сервісів виконується **до** `builder.Build()`. Відображення маршрутів — після нього. У файлі залишається один виклик `app.Run()`.

Тут не потрібні `_Host.cshtml`, `AddServerSideBlazor()` і `MapBlazorHub()` зі старого шаблону Blazor Server.

3. Що означає Interactive Server

HTML відображається у браузері, але обробники кнопок, C#-обчислення та операції серверного коду виконуються на сервері. Браузер надсилає події через з’єднання SignalR і отримує зміни інтерфейсу. Звичайний клік `@onclick` не є надсиланням HTML-форми методом POST.

Статично відрендерений компонент може показувати текст, але без інтерактивного режиму його `@onclick` не працюватиме. У цій роботі інтерактивність задається один раз для `Routes`, а сторінки

й їх дочірні компоненти її успадковують.

У **Components/App.razor** внесіть зміни відносно початкового шаблону:

```
--- a/Components/App.razor
+++ b/Components/App.razor
@@ -1,5 +1,5 @@
 <!DOCTYPE html>
-<html lang="en">
+<html lang="uk">

 <head>
     <meta charset="utf-8" />
@@ -9,11 +9,11 @@
     <link rel="stylesheet" href="@Assets["app.css"]" />
     <link rel="stylesheet" href="@Assets["CampusCafe.styles.css"]" />
     <ImportMap />
-    <HeadOutlet @rendermode="InteractiveServer" />
+    <HeadOutlet @rendermode="new InteractiveServerRenderMode(prerender: false)" />
 </head>

 <body>
-    <Routes @rendermode="InteractiveServer" />
+    <Routes @rendermode="new InteractiveServerRenderMode(prerender: false)" />
     <ReconnectModal />
     <script src="@Assets["_framework/blazor.web.js"]"></script>
 </body>
```

`prerender: false` вимикає попередній статичний рендеринг. Це спрощує навчальний сценарій: компоненти ініціалізуються після встановлення інтерактивного з'єднання. До цього вміст сторінки не показується. Це свідомий компроміс, а не вимога Blazor.

Без цього параметра попередній рендеринг увімкнений: сторінка може швидше показати початковий HTML, але ініціалізація може відбутися окремо для попереднього й інтерактивного рендерингу. Не варто записувати покупки в базу з `OnInitializedAsync()`.

`HeadOutlet` керує вмістом `<head>`, зокрема заголовком із `PageTitle`. `blazor.web.js` запускає клієнтську частину Blazor. `ReconnectModal` належить шаблону й показує стан відновлення з'єднання.

4. Перша модель та каталог без бази даних

Модель даних описує сутність предметної області. Один об'єкт `Drink` — один вид напою, а не вже налаштована позиція замовлення.

Адаптуйте зараз — одиниця каталогу. Визначте, що описує один запис: букет, фотопродукт, подарунковий набір чи інший товар. Поля `Id`, `Name`, `Description` та `BasePrice` придатні для всіх варіантів. `Icon` необов'язковий для власного каталогу; якщо його вилучаєте, приберіть також його початкові значення й використання в розмітці. Виберіть відповідну назву класу та використовуйте її замість `Drink` у наступних кроках.

Створіть папку **Models**, а в ній файл **Drink.cs**:

```
namespace CampusCafe.Models;

public class Drink
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Description { get; set; } = "";
    public string Icon { get; set; } = "☕";
    public decimal BasePrice { get; set; }
}
```

`Id` — ідентифікатор. `Name`, `Description` та `Icon` — рядки. `BasePrice` — ціна базової порції у гривнях. Для грошей використовуємо `decimal`; суфікс `m` у `65m` позначає саме такий числовий тип.

У кінець **Components/_Imports.razor** додайте:

```
--- a/Components/_Imports.razor
+++ b/Components/_Imports.razor
@@ -9,3 +9,4 @@
 @using CampusCafe
 @using CampusCafe.Components
 @using CampusCafe.Components.Layout
+@using CampusCafe.Models
```

Цей файл застосовує імпорти до компонентів у своїй папці та вкладених папках. Тепер можна писати `Drink`, а не `CampusCafe.Models.Drink`.

Адаптуйте зараз — перші два товари. У новій версії сторінки нижче одразу змініть заголовок, `Name`, `BasePrice` та `Icon` на свої дані. Поки достатньо двох записів у списку; повний каталог із шести товарів додасте в пам'яті наприкінці цієї роботи. Текст «Ваш вибір» має показувати назву вашого товару.

У **Components/Pages/Home.razor** замініть початковий вміст шаблону на перший навчальний каталог:

Було — вміст файла на попередньому етапі:

```
@page "/"

<PageTitle>Home</PageTitle>

<h1>Hello, world!</h1>

Welcome to your new app.
```

Стало — повний вміст після заміни:

```
@page "/"
<PageTitle>Меню</PageTitle>
<h1>Campus Café</h1>

@foreach (var drink in drinks)
{
    <p>
        @drink.Icon @drink.Name — @drink.BasePrice грн
        <button @onclick="() => selectedName = drink.Name">Обрати</button>
    </p>
}
<p>Ваш вибір: @selectedName</p>

@code {
    private string selectedName = "ще не обрано";
    private List<Drink> drinks = new()
    {
        new Drink { Id = 1, Name = "Капучино", BasePrice = 65m },
        new Drink { Id = 2, Name = "Латте", BasePrice = 70m }
    };
}
```

`@page "/"` пов'язує компонент із головною адресою. `@foreach` створює повторювану розмітку. `@drink.Name` виводить значення, а `@code` містить поля й методи класу компонента.

`@onclick="() => selectedName = drink.Name"` передає обробник події, а не виконує присвоєння під час побудови розмітки. Після кліку Blazor виконує його і повторно рендерить компонент. Викликати `StateHasChanged()` у цьому прикладі вручну не потрібно.

Контроль: на сторінці два напої, а клік змінює текст «Ваш вибір» без перезавантаження браузера.

Перевірте: до натискання кнопки передбачте, яке поле зміниться. Натисніть її та знайдіть у коді присвоєння нового значення. Чому для цього не потрібні форма, `OnPost()` або запит до SQLite?

5. Модель налаштування і позиція замовлення

Не змінюйте об'єкт із каталогу, коли користувач налаштовує покупку: однакова кава може бути додана з різними опціями. Для цього потрібні окремі об'єкти.

Тип	Що містить	Де зберігається
`Drink`	Назва, опис, базова ціна	Список у компоненті Home
`DrinkDraft`	Поточний вибір у діалозі	Тимчасово в компоненті
`OrderLine`	Підтверджені опції, кількість і ціна	Останній підтверджений вибір у Home

Адаптуйте зараз — параметри товару. Спочатку узгодьте назви та одиниці вимірювання в моделях, діалозі й підтвердженому результаті. Для теми «Майстерня букетів» відповідність може бути такою:

У демонстрації кав’ярні	У застосунку букетів	Де змінити
`Drink`, `DrinkId`, `DrinkName`	`Bouquet`, `BouquetId`, `BouquetName`	Модель каталогу, `OrderLine`, параметри компонентів і методів
`DrinkDraft`	`BouquetDraft`	Модель чернетки та поле `draft` у діалозі
`Volume`: 250, 350, 450	`FlowerCount`: 5, 9, 15	Чернетка, `switch`, діалог та збережена позиція
`MilkKind.Regular`, `MilkKind.Oat`	`PackagingKind.Standard`, `PackagingKind.Gift`	Перелік варіантів, доплата та `<select>`
`Milk`: «звичайне», «вівсяне»	`Packaging`: «звичайне», «подарункове»	Текст підтвердженої позиції
«мл», «молоко»	«квітів», «пакування»	Підписи діалогу й підтвердженого результату

`Quantity`, `UnitPrice`, `Total` та ідентифікатор позиції можна залишити без зміни. `Quantity` у цьому варіанті — кількість букетів, а не квітів у кожному букеті. Виконайте ці зміни **до першого створення власної бази**.

Створіть **Models/OrderLine.cs**:

```
namespace CampusCafe.Models;

public class OrderLine
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public int DrinkId { get; init; }
    public string DrinkName { get; init; } = "";
    public int Volume { get; init; }
    public string Milk { get; init; } = "";
    public int Quantity { get; init; }
    public decimal UnitPrice { get; init; }
    public decimal Total => UnitPrice * Quantity;
}
```

`Guid.NewGuid()` створює окремий ідентифікатор позиції. Два однакові напої можуть мати різні `Id` позицій. `init` дозволяє задавати значення під час створення об’єкта, але не змінювати їх звичайним присвоєнням пізніше. Це захищає підтверджену позицію від випадкового редагування чернеткою.

`Total => UnitPrice * Quantity` — обчислювана властивість. Вона не зберігає другу копію суми, а рахує її при зверненні.

Адаптуйте зараз — правило ціни. Змініть значення за замовчуванням, допустимі варіанти, гілки `switch` і доплати. Одночасно адаптуйте `CreateLine`: до підтвердженої позиції мають потрапити саме ваші опції.

Наприклад, для букета базова ціна 300 грн за 5 квітів; доплата за 9 квітів — 160 грн, за подарункове пакування — 50 грн. Два такі букети: `(300 + 160 + 50) × 2 = 1020 грн`. Це лише правило навчального магазину, а не вимога до всіх тем. Якщо у вашій темі ціна множиться на площу чи довжину, змініть формулу, а не лише назви змінних.

Створіть **Models/DrinkDraft.cs**:

```
namespace CampusCafe.Models;

public enum MilkKind { Regular, Oat }

public class DrinkDraft
{
    public int Volume { get; set; } = 250;
    public MilkKind Milk { get; set; } = MilkKind.Regular;
    public int Quantity { get; set; } = 1;

    public decimal GetUnitPrice(Drink drink)
    {
        decimal volumeExtra = Volume switch
        {
            250 => 0m,
            350 => 15m,
            450 => 25m,
            _ => throw new InvalidOperationException("Неприпустимий об’єм.")
        };
        decimal milkExtra = Milk switch
        {
            MilkKind.Regular => 0m,
            MilkKind.Oat => 20m,
            _ => throw new InvalidOperationException("Неприпустимий вид молока.")
        };
        return drink.BasePrice + volumeExtra + milkExtra;
    }

    public OrderLine CreateLine(Drink drink)
    {
        if (Quantity is < 1 or > 10)
            throw new InvalidOperationException("Кількість має бути від 1 до 10.");

        return new OrderLine
        {
            DrinkId = drink.Id,
            DrinkName = drink.Name,
            Volume = Volume,
            Milk = Milk == MilkKind.Oat ? "вівсяне" : "звичайне",
            Quantity = Quantity,
            UnitPrice = GetUnitPrice(drink)
        };
    }
}
```

`enum MilkKind` задає обмежений набір варіантів. `switch` обирає доплату. Гілка `_` відхиляє непідтримуване значення. HTML-обмеження зручні для користувача, але правила перевіряються також у C#.

Формула:

```
Ціна одного = базова ціна + доплата за об’єм + доплата за молоко
Сума позиції = ціна одного × кількість
```

Для капучино: `65 + 15 + 20 = 100 грн`. Дві порції коштують `200 грн`. Метод `CreateLine` створює новий об’єкт підтвердженої позиції. Скасування діалогу його не викликає.

Перевірте: обчисліть свою позицію вручну й через метод чернетки. Змініть один параметр. Чи змінилася ціна так, як ви очікували? Після створення позиції змініть чернетку ще раз: уже підтверджена позиція не повинна змінитися.

6. Макет і компонент форматування ціни

Макет — спільна оболонка сторінок. `@Body` позначає місце, куди маршрутизатор вставляє поточну сторінку. Поки що назву магазину записано безпосередньо в макеті; спільні налаштування введемо в лабораторній 8.

Адаптуйте зараз: назву, символ магазину й підвал. Залиште `MainLayout.razor.css` та `ReconnectModal.*` із шаблону.

Components/Layout/MainLayout.razor — було:

```
@inherits LayoutComponentBase

@Body

<div id="blazor-error-ui" data-nosnippet>
    An unhandled error has occurred.
    <a href="." class="reload">Reload</a>
    <span class="dismiss">🗙</span>
</div>
```

Стало — повний файл:

```
@inherits LayoutComponentBase

<header class="site-header">
    <a class="brand" href="/">☕ Campus Café</a>
    <nav aria-label="Основна навігація">
        <NavLink href="/" Match="NavLinkMatch.All">Каталог</NavLink>
    </nav>
</header>
<main class="page-shell">@Body</main>
<footer class="site-footer">Навчальний проєкт LNTU · Ціни у гривнях</footer>

<div id="blazor-error-ui" data-nosnippet>
    Роботу застосунку перервано.
    <a href="." class="reload">Перезавантажити сторінку</a>
```

```
    <span class="dismiss">×</span>
</div>
```

Створіть папку `Components/Shared` і новий файл **MoneyLabel.razor**:

```
@using System.Globalization
<span>@Amount.ToString("N2", CultureInfo.GetCultureInfo("uk-UA")) ₴</span>

@code {
    [Parameter] public decimal Amount { get; set; }
}
```

`Amount` — звичайний параметр компонента. `N2` задає два десяткові знаки; `uk-UA` — український формат числа. Додавання символу гривні не конвертує валюту: всі суми вже задані у гривнях.

У кінець **Components/_Imports.razor**, після імпорту моделей, додайте:

```
 @using CampusCafe.Models
+@using CampusCafe.Components.Shared
```

Імпорт `CampusCafe.Services` поки не потрібний: такої папки ще немає.

7. Картка товару: параметри та подія вибору

Адаптуйте зараз — картки й зображення. Чашки та `CupViewBox` нижче належать лише кавовому прикладу. Для власної теми підготуйте відповідні ілюстрації й змініть спосіб їх вибору. При заміні назв товарів існуючий `switch` не знайде збігів і повертатиме одну чашку з гілки `_` — це не автоматичне добирання зображення.

Для окремих файлів зображень можна додати до своєї моделі поле `ImageUrl`, задати його в локальному списку Home; у наступній роботі — в seed та замінити `<svg>...</svg>` на `<img src="@Drink.ImageUrl" alt="@Drink.Name" />` із відповідним CSS. У цьому фрагменті `Drink` — ім'я параметра демонстрації; узгодьте його зі своїм компонентом. Тоді `CupViewBox` більше не потрібний. Бази даних поки немає; перед її створенням у наступній роботі узгодьте остаточну модель. Інший варіант — словник локальних шляхів у компоненті, який не змінює базу.

Шлях до локального файла має починатися з `images/`, а не з `wwwroot/`. Для оприлюдненого застосунку використовуйте власні ілюстрації або ресурси з дозволом на таке використання.

Перед створенням картки додайте ресурси до проєкту:

1. Створіть папки `wwwroot/images` та `wwwroot/fonts`.
2. Збережіть ілюстрацію з чашками як `wwwroot/images/coffee-cups.jpg` без зміни розміру (1024 × 1386).
3. Скопіюйте з готового демопроєкту файли `quicksand-v8-latin-regular.woff2`, `quicksand-v8-latin-700.woff2` та `OFL-Quicksand.txt` до `wwwroot/fonts`. Оригінали шрифтів містяться в ресурсах Blazing Pizza.

Ресурси в готовому демопроєкті вже наявні. Посилання в розмітці починаються з `images/`, а не з `wwwroot/`: ця папка є коренем статичних файлів вебсайту.

Створіть **Components/Shared/DrinkCard.razor**:

```
<article class="drink-card">
    <div class="drink-art">
        <svg viewBox="@CupViewBox" aria-hidden="true" focusable="false"
preserveAspectRatio="xMidYMid meet">
            <image href="images/coffee-cups.jpg" width="1024" height="1386" />
        </svg>
        <strong class="price-badge">від <MoneyLabel Amount="Drink.BasePrice" /></strong>
    </div>
    <div class="card-body">
        <h2>@Drink.Name</h2>
        <p>@Drink.Description</p>
        <div class="card-footer">
            <span>Налаштуйте свій напій</span>
            <button type="button" @onclick="SelectAsync" aria-label="@($"Налаштувати
{Drink.Name}")">Обрати</button>
        </div>
    </div>
</article>

@code {
    [Parameter, EditorRequired] public Drink Drink { get; set; } = default!;
    [Parameter] public EventCallback<Drink> OnSelect { get; set; }
    private string CupViewBox => Drink.Name switch
    {
        "Капучино" => "520 15 235 225",
        "Латте" => "300 15 235 225",
        "Мокачино" => "520 295 235 225",
        "Какао" => "60 1100 235 225",
        "Матча-латте" => "60 295 235 225",
        _ => "300 295 235 225"
    };
    private Task SelectAsync() => OnSelect.InvokeAsync(Drink);
}
```

Вкладений SVG використовує оригінальний JPEG як зображення, а `viewBox` визначає видиму прямокутну ділянку: `x y ширина висота`. Властивість `CupViewBox` вибирає чашку за назвою напою; SVG не змінює файл і не обрізає його на диску. Для нових назв додайте відповідну гілку `switch`. Чашки є умовними ілюстраціями; для матча й какао вони не відтворюють точний вигляд рецепта. `aria-hidden="true"` приховує декоративне зображення від читача екрана, оскільки назву напою вже містить заголовок картки.

Картка отримує об'єкт `Drink` та показує його. `[EditorRequired]` допомагає інструментам помітити відсутній параметр, але не є повноцінною перевіркою даних під час виконання.

`EventCallback<Drink>` передає подію вибору разом із даними напою. Картка не вирішує, як створити кошик чи відкрити діалог: вона лише викликає `OnSelect.InvokeAsync(Drink)`. Реакцію визначить батьківська сторінка.

```
Home ── Drink ──→ DrinkCard
Home ←── OnSelect(drink) ── DrinkCard
```

Метод повертає `Task`, щоб Blazor міг очікувати асинхронний обробник. Для обробників подій не використовуйте `async void`.

8. Діалог налаштування та прив'язування даних

Адаптуйте зараз — інтерфейс параметрів. Узгодьте `@bind` із властивостями своєї чернетки. Змініть написи «Об'єм» і «Молоко», значення `<option>` та показані доплати. Наприклад, `value="9"` може означати 9 квітів, а не 350 мл; воно має відповідати дозволеній гілці `switch`.

Зміна напису «+20 ₴» у HTML **не змінює обчислення** в `GetUnitPrice`. І навпаки: після зміни формули перевірте підписи, щоб користувач не бачив іншої доплати. Кнопки підтвердження й скасування та механізм `EventCallback` залишаються тими самими.

Створіть **Components/Shared/ConfigureDrinkDialog.razor**:

```
<div class="modal-backdrop">
    <section class="configure-dialog" role="dialog" aria-modal="true" aria-
labelledby="dialog-title">
        <header>
            <span class="eyebrow">Ваш напій · ваш смак</span>
            <h2 id="dialog-title">@Drink.Name</h2>
            <p>@Drink.Description</p>
        </header>
        <div class="dialog-fields">
            <label for="volume">Об'єм</label>
            <select id="volume" @bind="draft.Volume">
                <option value="250">250 мл — базова ціна</option>
                <option value="350">350 мл — +15 ₴</option>
                <option value="450">450 мл — +25 ₴</option>
            </select>

            <label for="milk">Молоко</label>
            <select id="milk" @bind="draft.Milk">
                <option value="@MilkKind.Regular">Звичайне — без доплати</option>
                <option value="@MilkKind.Oat">Вівсяне — +20 ₴</option>
            </select>

            <label for="quantity">Кількість: @draft.Quantity</label>
            <input id="quantity" type="range" min="1" max="10" step="1"
                   @bind="draft.Quantity" @bind:event="oninput" />

            <div class="price-preview" aria-live="polite">
                <span>За один: <MoneyLabel Amount="draft.GetUnitPrice(Drink)" /></span>
                <strong>Разом: <MoneyLabel Amount="@(draft.GetUnitPrice(Drink) *
draft.Quantity)" /></strong>
            </div>
        </div>
        <footer>
            <button type="button" class="secondary" @onclick="OnCancel">Скасувати</button>
```

```
            <button type="button" @onclick="ConfirmAsync">Підтвердити вибір</button>
        </footer>
    </section>
</div>

@code {
    [Parameter, EditorRequired] public Drink Drink { get; set; } = default!;
    [Parameter] public EventCallback OnCancel { get; set; }
    [Parameter] public EventCallback<OrderLine> OnConfirm { get; set; }
    private readonly DrinkDraft draft = new();

    private Task ConfirmAsync() => OnConfirm.InvokeAsync(draft.CreateLine(Drink));
}
```

`@bind="draft.Volume"` пов'язує вибір у `<select>` із властивістю C#. Коли користувач обирає об'єм, змінюється `draft.Volume` і компонент рендериться знову. Це Blazor-прив'язування: тут не використовуються `asp-for` і `[BindProperty]` із Razor Pages.

Для повзунка `@bind:event="oninput"` оновлює кількість у процесі переміщення, а не лише після завершення зміни. `@bind` уже використовує подію, тому не додавайте другий обробник тієї самої події без зміни підходу до прив'язування.

`GetUnitPrice(Drink)` обчислюється під час рендерингу. Це не спеціальна «реактивна функція»: Blazor повторно виконує розмітку після події і бачить новий результат.

`OnCancel` не має додаткових даних. `OnConfirm` передає готовий `OrderLine`. Поки не натиснуто «Підтвердити вибір», змінюється лише чернетка.

Діалог є окремим компонентом, але не має `@page`: відкрити його прямою адресою не можна. Його присутністю в дереві керуватиме `@if` у `Home`.

Перевірте: відкрийте налаштування, змініть дві опції та скасуйте. Які дані залишилися незмінними? Повторно відкрийте той самий товар і поясніть початкові значення чернетки.

9. Поєднання компонентів і підтверджений результат

До цього моменту картка й діалог створені, але ще не підключені до Home. Тепер замініть тимчасову сторінку з двома записами.

Адаптуйте зараз: внесіть усі шість власних товарів, унікальні `Id`, описи та ціни. Узгодьте назви моделей, компонентів і одиниці вимірювання з попередніми кроками.

Components/Pages/Home.razor — було:

```
@page "/"
<PageTitle>Меню</PageTitle>
<h1>Campus Café</h1>

@foreach (var drink in drinks)
{
    <p>
```

```
        @drink.Icon @drink.Name — @drink.BasePrice грн
        <button @onclick="() => selectedName = drink.Name">Обрати</button>
    </p>
}
<p>Ваш вибір: @selectedName</p>

@code {
    private string selectedName = "ще не обрано";
    private List<Drink> drinks = new()
    {
        new Drink { Id = 1, Name = "Капучино", BasePrice = 65m },
        new Drink { Id = 2, Name = "Латте", BasePrice = 70m }
    };
}
```

Стало — повний файл:

```
@page "/"
<PageTitle>Каталог · Campus Café</PageTitle>

<section class="hero">
    <p class="eyebrow">LNTU · кава між парами</p>
    <h1>Campus Café</h1>
    <p>Оберіть напій і налаштуйте його. Поки що зберігаємо лише останній вибір.</p>
</section>

<div class="catalog">
    @foreach (var drink in drinks)
    {
        <DrinkCard @key="drink.Id" Drink="drink" OnSelect="SelectDrink" />
    }
</div>

<section aria-live="polite">
    <h2>Останній підтверджений вибір</h2>
    @if (lastSelection is null)
    {
        <p>Ви ще нічого не підтвердили.</p>
    }
    else
    {
        <p>@lastSelection.DrinkName · @lastSelection.Volume мл · @lastSelection.Milk молоко
· @lastSelection.Quantity шт.</p>
        <strong><MoneyLabel Amount="lastSelection.Total" /></strong>
    }
</section>

@if (selectedDrink is not null)
{
    <ConfigureDrinkDialog @key="selectedDrink.Id" Drink="selectedDrink"
                          OnCancel="CloseDialog" OnConfirm="ConfirmDrink" />
}

@code {
    private Drink? selectedDrink;
    private OrderLine? lastSelection;
```

```
    private readonly List<Drink> drinks = new()
    {
        new() { Id = 1, Name = "Капучино", Description = "Еспресо, молоко та ніжна пінка.",
BasePrice = 65m },
        new() { Id = 2, Name = "Латте", Description = "М’який смак і більше молока.",
BasePrice = 70m },
        new() { Id = 3, Name = "Мокачино", Description = "Кава з шоколадом.", BasePrice =
85m },
        new() { Id = 4, Name = "Матча-латте", Description = "Зелений чай матча з молоком.",
BasePrice = 90m },
        new() { Id = 5, Name = "Какао", Description = "Шоколадний напій без кави.",
BasePrice = 60m },
        new() { Id = 6, Name = "Чай-латте", Description = "Пряний чай із молоком.",
BasePrice = 75m }
    };

    private void SelectDrink(Drink drink) => selectedDrink = drink;
    private void CloseDialog() => selectedDrink = null;
    private void ConfirmDrink(OrderLine line)
    {
        lastSelection = line;
        CloseDialog();
    }
}
```

`selectedDrink` керує видимістю діалогу, а `lastSelection` містить підтверджений результат. Це різні дані. Скасування обнуляє лише `selectedDrink` і не зачіпає попередній результат.

`@key` допомагає зіставляти компонент із товаром. Після закриття діалог прибирається з дерева, тому повторне відкриття створює нову чернетку.

```
Home → DrinkCard: товар через Parameter
DrinkCard → Home: OnSelect із вибраним товаром
Home → ConfigureDrinkDialog: вибраний товар
@bind → DrinkDraft → розрахунок суми
ConfigureDrinkDialog → Home: OnConfirm із новим OrderLine
Home → lastSelection → показ підтвердженого результату
```

Назву `OrderLine` використано наперед: зараз це одна підтверджена позиція; у лабораторній 8 такі самі об’єкти утворять кошик. Зміна чернетки не змінює вже створений об’єкт позиції.

10. Оформлення

Повністю замініть вміст **wwwroot/app.css** наведеним нижче. Старі стилі шаблону тут не повторюються. Після перевірки макета адаптуйте палітру, шрифти й оформлення карток до своєї тематики.

Новий повний вміст `wwwroot/app.css`:

```
@font-face { font-family:Quicksand; font-style:normal; font-weight:400; font-display:swap;
src:url('fonts/quicksand-v8-latin-regular.woff2') format('woff2'); }
@font-face { font-family:Quicksand; font-style:normal; font-weight:700; font-display:swap;
```

```
src:url('fonts/quicksand-v8-latin-700.woff2') format('woff2'); }
:root { --red:#c00000; --cart:#2b2b2b; --green:#087e12; --yellow:#ffc24a; --cream:#fff2cc;
--text:#292929; }
* { box-sizing:border-box; }
html { min-height:100%; }
body { margin:0; background:linear-gradient(#fff 25%,#bdbdbd 100%) fixed; color:var(--
text); font-family:Quicksand,'Trebuchet MS',Arial,sans-serif; line-height:1.5; }
a { color:inherit; }
button,select,input { font:inherit; }
button,.button-link { background:var(--green); color:#fff; border:0; border-radius:5px;
padding:.65rem 1rem; cursor:pointer; text-decoration:none; font-weight:700; }
button:hover,.button-link:hover { background:#05600d; }
button:focus-visible,a:focus-visible,select:focus-visible,input:focus-visible { outline:3px
solid #e4a321; outline-offset:4px; }
.secondary { background:#666; color:white; }
.secondary:hover { background:#505050; }
.site-header { min-height:80px; padding:0 3%; display:flex; justify-content:space-between;
align-items:stretch; gap:25px; color:#fff; background:linear-
gradient(#0005,transparent),var(--red); box-shadow:0 2px 5px #0005; }
.brand { font-size:1.7rem; font-weight:700; text-decoration:none; display:flex; align-
items:center; gap:12px; white-space:nowrap; }
.brand span { font-size:2.5rem; }
nav { display:flex; align-items:stretch; }
nav a { display:flex; align-items:center; text-align:center; padding:20px 25px; text-
decoration:none; font-size:.9rem; font-weight:700; text-transform:uppercase;
background:#0001; }
nav a:hover { background:#ffffff30; }
nav a.active { background:#ffffff26; color:var(--cream); box-shadow:inset 0 -4px var(--
yellow); }
.page-shell { max-width:1600px; margin:auto; padding:28px 3% 48px; }
.hero { margin-bottom:26px; }
.eyebrow { text-transform:uppercase; font-size:.72rem; letter-spacing:.13em; font-
weight:700; color:#a72323; }
.hero .eyebrow { margin:0; }
h1,h2 { font-family:Bahnschrift,'Arial Narrow',Arial,sans-serif; }
h1 { font-size:clamp(2rem,4vw,3rem); font-weight:700; line-height:1.1; margin:10px 0; }
h1 em { font-style:normal; }
h1:focus { outline:none; }
.hero>p:last-child { margin-bottom:0; color:#555; }
h2 { line-height:1.25; }
.shop-grid { display:grid; grid-template-columns:minmax(0,1fr) 320px; gap:26px; align-
items:start; }
.section-heading { display:flex; align-items:center; justify-content:space-between;
gap:12px; margin-bottom:18px; }
.section-heading h2 { margin:0; font-size:1.45rem; }
.section-heading span { font-size:.8rem; color:#555; }
.catalog { display:grid; grid-template-columns:repeat(3,minmax(0,1fr)); gap:22px; }
.drink-card { border-radius:8px; overflow:hidden; background:#343434; box-shadow:0 3px 6px
#0005; transition:transform .15s ease; }
.drink-card:hover { transform:translateY(-3px); }
.drink-art { position:relative; display:flex; align-items:center; justify-content:center;
height:195px; background:#f4ebdd; overflow:hidden; }
.drink-art svg { display:block; width:auto; height:100%; aspect-ratio:235 / 225; flex-
shrink:0; overflow:hidden; }
.price-badge { position:absolute; right:10px; bottom:9px; background:var(--green);
color:white; padding:4px 9px; font-size:1.1rem; border-radius:4px; box-shadow:0 2px 3px
#0003; }
.card-body { padding:17px; color:var(--cream); background:linear-gradient(#444,#2b2b2b); }
```

```
.card-body h2 { margin:0; font-size:1.2rem; text-transform:uppercase; color:#fff; }
.card-body p { margin:9px 0 16px; min-height:3em; font-size:.87rem; }
.card-footer { display:flex; justify-content:space-between; gap:10px; align-items:center; }
.card-footer>span { font-size:.68rem; color:#d3d3d3; }
.card-footer button { background:var(--yellow); color:#352600; padding:.5rem .85rem; }
.card-footer button:hover { background:#ffd477; }
.cart { background:var(--cart); color:#fff; border-radius:8px; overflow:hidden;
padding:25px; position:sticky; top:20px; box-shadow:0 3px 6px #0004; min-height:355px; }
.cart .eyebrow { color:#e2b66d; }
.cart h2 { display:flex; align-items:center; justify-content:space-between; text-
transform:uppercase; }
.count { background:#555; color:var(--cream); border-radius:50%; padding:5px 12px; font-
size:.85rem; }
.empty-cart { padding:35px 0; color:#d0d0d0; font-size:.9rem; text-align:center; }
.cart-items { list-style:none; padding:0; margin:0; }
.cart-items li { padding:16px; background:#383838; border-radius:4px; margin-bottom:12px; }
.cart-items p { font-size:.78rem; color:#d4d4d4; }
.cart-line-footer,.cart-total { display:flex; justify-content:space-between; gap:10px;
align-items:center; }
.cart-total { margin:24px -25px 18px; padding:20px 25px; background:#bf3434; }
.cart-total strong { font-size:1.3rem; white-space:nowrap; }
.text-button { border:0; background:transparent; color:#ffd991; padding:0; font-
size:.78rem; text-decoration:underline; }
.text-button:hover { background:transparent; color:white; }
.full-width { width:100%; }
.cart-note { font-size:.72rem; color:#c5c5c5; margin:22px 0 0; }
.modal-backdrop { position:fixed; inset:0; background:#0009; display:grid; place-
items:center; padding:24px; z-index:100; overflow-y:auto; }
.configure-dialog { width:min(100%,570px); max-height:calc(100dvh - 48px); overflow:auto;
border-radius:8px; background:#fff; box-shadow:0 12px 55px #0007; }
.configure-dialog header { background:#444; color:#fff; padding:25px 28px; }
.configure-dialog .eyebrow { color:#ffd991; }
.configure-dialog h2 { margin:8px 0; font-size:1.65rem; text-transform:uppercase; }
.configure-dialog header p { margin:0; color:var(--cream); }
.dialog-fields { display:grid; gap:10px; padding:20px 28px 25px; }
.dialog-fields label { font-weight:700; margin-top:6px; }
select { width:100%; background:#fff; padding:10px; border:1px solid #aaa; border-
radius:4px; color:#292929; }
input[type=range] { width:100%; accent-color:#c94242; }
.price-preview { display:grid; gap:6px; padding-top:16px; }
.price-preview strong { font-size:1.3rem; }
.configure-dialog footer { display:flex; justify-content:space-between; gap:12px;
background:#eee; padding:20px 28px; }
.site-footer { background:#292929; padding:20px 3%; text-align:center; color:#d0d0d0; font-
size:.8rem; }
.valid.modified:not([type=checkbox]) { outline:1px solid #26b050; }
.invalid { outline:1px solid #e50000; }
.validation-message { color:#b32121; }
@media (max-width:1250px) { .catalog { grid-template-columns:repeat(2,minmax(0,1fr)); } }
@media (max-width:950px) { .shop-grid { grid-template-columns:minmax(0,1fr) 280px;
gap:18px; } .catalog { gap:16px; } .card-footer { align-items:stretch; flex-
direction:column; } .card-body { padding:13px; } }
@media (max-width:760px) { .shop-grid { grid-template-columns:1fr; } .cart {
position:static; } .site-header { padding:12px 4% 0; align-items:center; flex-
direction:column; gap:10px; } nav { width:100%; justify-content:center; } nav a {
padding:12px 18px; } .section-heading { align-items:flex-start; flex-direction:column; } }
@media (max-width:440px) { .catalog { grid-template-columns:1fr; } .card-footer { flex-
direction:row; align-items:center; } .drink-art { height:205px; } .configure-dialog footer
```

```
{ flex-direction:column-reverse; } }
@media (prefers-reduced-motion:reduce) { .drink-card { transition:none; } .drink-card:hover
{ transform:none; } }
```

Адаптуйте зараз — візуальна тема. Після запуску наведеного CSS змініть основні кольори в `:root`, тло карток, шрифт заголовків та подання власних ілюстрацій. Назви CSS-класів (`.catalog`, `.cart`, `.configure-dialog`) можуть залишитися тими самими: вони описують роль елемента, а не товар. Перевірте контраст тексту й поведінку вузького екрана. Якщо використовуєте `<img>` замість SVG, задайте йому розміри та `object-fit` відповідно до пропорцій ваших зображень.

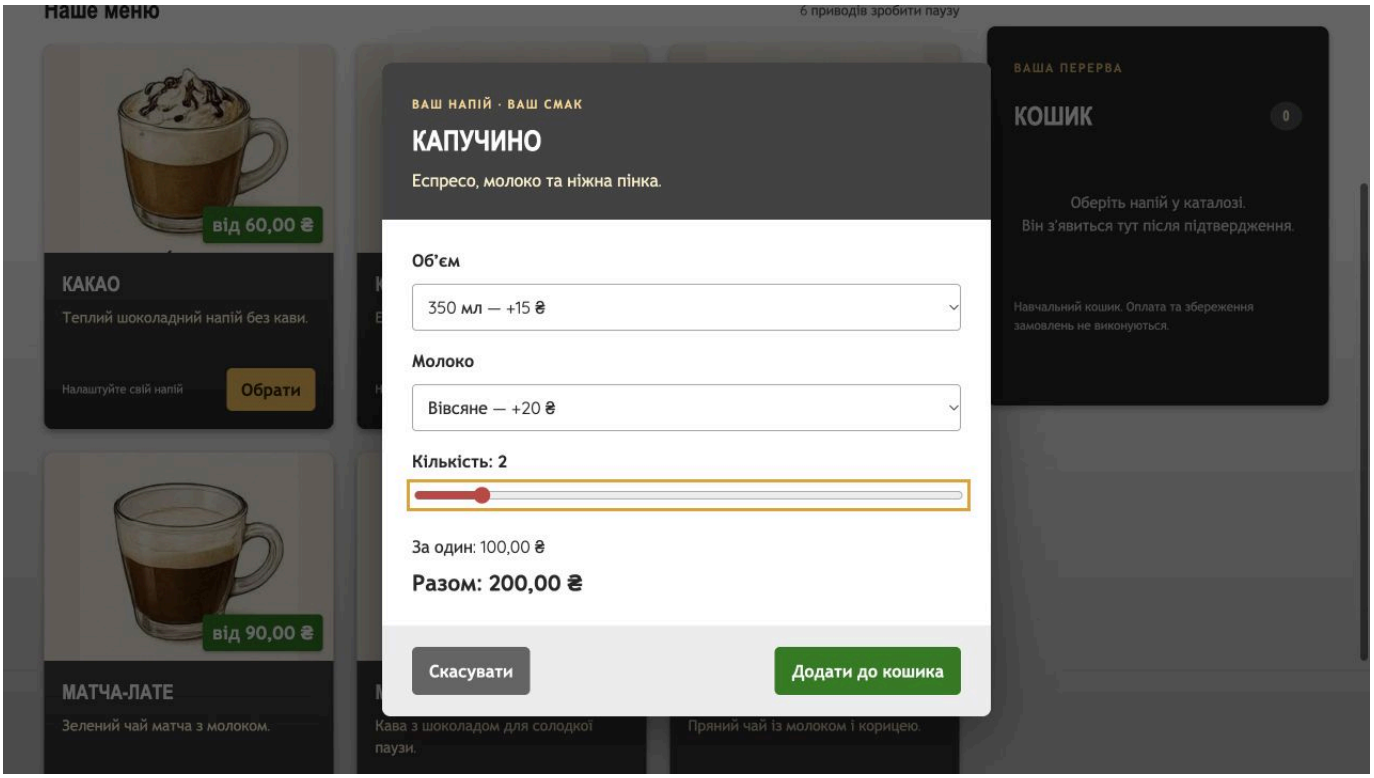
CSS Grid створює каталог. Стилі кошика й двоколонкового макета підготовлені для наступної роботи; зараз ці елементи не відображаються. Медіазапити перебудовують макет для вузького екрана. `.modal-backdrop` затемнює фон, а `.configure-dialog` оформлює вікно налаштувань.

Оформлення наслідує Blazing Pizza: червона градієнтна шапка, сіре тло, темні картки й кошик, зелені цінники та жовті кнопки вибору. Кошик має червоний блок загальної суми, а діалог — темну шапку та світлу форму.

Шрифт Quicksand підключений локально через `@font-face`. Ці WOFF2-файли містять латинські символи; для українського тексту використовуються Trebuchet MS або Arial. Заголовки мають набір `Bahnschrift, 'Arial Narrow', Arial`, подібний до зразка. Фотоілюстрація також локальна — під час роботи сайт не залежить від Pinterest.

Властивість `Icon` залишається в моделі й початковій вправі, але фінальна картка показує чашку з JPEG замість емодзі. Бази даних у цій роботі ще немає. Для власного каталогу можна додати `ImageUrl` і використовувати окремі файли; перед створенням БД у наступній роботі узгодьте ці дані з моделлю.

Оформлення не визначає бізнес-логіку: зміна кольору картки не змінює ціну. Для повноцінного виробничого модального вікна додатково реалізують перенесення й утримання клавіатурного фокуса, закриття Escape та повернення фокуса до кнопки відкриття. У цій вправі діалог демонструє передавання параметрів і подій.



На ілюстрації показано діалог завершеного демо. У цій роботі ваша кнопка має напис «Підтвердити вибір», а панелі кошика ще немає.

11. Структура та призначення файлів

Основні файли наприкінці роботи; допоміжні файли шаблону залишаються без змін.

```
CampusCafe/
├── Program.cs                        початковий запуск, без власних сервісів
├── Models/
│   ├── Drink.cs                      товар каталогу
│   ├── DrinkDraft.cs                 опції та формула ціни
│   └── OrderLine.cs                  підтверджена позиція
├── Components/
│   ├── App.razor                     інтерактивність і HTML-оболонка
│   ├── Routes.razor                  маршрутизатор шаблону
│   ├── _Imports.razor                імпорти Models і Shared
│   ├── Layout/MainLayout.razor       спільна оболонка
│   ├── Pages/Home.razor              каталог у пам'яті та останній вибір
│   └── Shared/
│       ├── MoneyLabel.razor           форматування суми
│       ├── DrinkCard.razor            картка і подія вибору
│       └── ConfigureDrinkDialog.razor  налаштування і підтвердження
└── wwwroot/                         CSS, images, fonts
```

Створено шість нових файлів: три моделі й три спільні компоненти. Змінено `App.razor`, `_Imports.razor`, `MainLayout.razor`, `Home.razor` і `app.css`. Власних `Data`, `Services` та `cafe.db` ще немає.

12. Перевірка, захист і підсумок

Виконайте `dotnet build`, а потім `dotnet run --urls http://localhost:5127`.

Перевірка	Очікуваний результат
Відкрити сторінку	Шість карток власних товарів
Капучино, 350 мл, вівсяне молоко, 2 шт.	100 грн за один, 200 грн разом
Підтвердити	Під каталогом з'являється сформована позиція
Відкрити інший товар і скасувати	Попередній підтверджений результат не змінюється
Підтвердити інший товар	Попередній результат замінюється, а не накопичується
Повторно відкрити діалог	Чернетка має початкові значення
Повністю перезавантажити сторінку	Останній вибір зникає, список відтворюється з коду

Для власної теми замініть контрольні числа своїми. Якщо кнопки не працюють, перевірте Interactive Server. Якщо компонент не знайдено — його назву й `_Imports.razor`. Якщо зображення відсутні — файли в `wwwroot/images` і відносні URL. Якщо сума неправильна — формулу, а не CSS.

Запам'ятайте: `Drink` описує товар; `DrinkDraft` — змінюваний вибір; `OrderLine` — його підтверджений знімок. `[Parameter]` передає дані дочірньому компоненту, `EventCallback` повідомляє батьківський, `@bind` оновлює властивість із поля введення. `@if` додає або прибирає діалог, а не просто приховує його CSS.

Під час захисту поясніть повний шлях одного натискання. Змініть одну доплату й перевірте результат. Покажіть, чому скасування не змінює останній вибір.

Збережіть цю версію окремим комітом або резервною копією. Наступна робота — Лабораторна 8: замість одного результату з'явиться спільний кошик, а каталог буде перенесено до SQLite.

Джерела для поглибленого вивчення

- Режими рендерингу Blazor.
- Використання EF Core у серверному Blazor.
- Прив'язування даних у компонентах.
- Обробка подій і EventCallback.
- Каскадні значення та параметри.

Джерела оформлення та ресурсів: Blazing Pizza, ілюстрація чашок. Відомості про шрифти й походження ілюстрації містяться в `ASSETS.md` комплекту демопроєкту.